

9.5 Machine Translation and the Dataset

We have used RNNs to design language models, which are key to natural language processing. Another flagship benchmark is *machine translation*, a central problem domain for *sequence transduction* models that transform input sequences into output sequences. Playing a crucial role in various modern AI applications, sequence transduction models will form the focus of the remainder of this chapter and [Chapter 10](#). To this end, this section introduces the machine translation problem and its dataset that will be used later.

Machine translation refers to the automatic translation of a sequence from one language to another. In fact, this field may date back to 1940s soon after digital computers were invented, especially by considering the use of computers for cracking language codes in World War II. For decades, statistical approaches had been dominant in this field ([Brown et al., 1988, 1990](#)) before the rise of end-to-end learning using neural networks. The latter is often called *neural machine translation* to distinguish itself from *statistical machine translation* that involves statistical analysis in components such as the translation model and the language model.

Emphasizing end-to-end learning, this book will focus on neural machine translation methods. Different from our language model problem in [Section 8.3](#) whose corpus is in one single language, machine translation datasets are composed of pairs of text sequences that are in the source language and the target language, respectively. Thus, instead of reusing the preprocessing routine for language modeling, we need a different way to preprocess machine translation datasets. In the following, we show how to load the preprocessed data into minibatches for training.

```
import os
import torch
from d2l import torch as d2l
```

9.5.1 Downloading and Preprocessing the Dataset

To begin with, we download an English-French dataset that consists of **bilingual sentence pairs from the Tatoeba Project**¹¹⁴. Each line in the dataset is a tab-delimited pair of an English text sequence and the translated French text sequence. Note that each text sequence can be just one sentence or a paragraph of multiple sentences. In this machine translation problem where English is translated into French, English is the *source language* and French is the *target language*.

```
#@save
d2l.DATA_HUB['fra-eng'] = (d2l.DATA_URL + 'fra-eng.zip',
                           '94646ad1522d915e7b0f9296181140edcf86a4f5')

#@save
def read_data_nmt():
    """Load the English-French dataset."""
    data_dir = d2l.download_extract('fra-eng')
    with open(os.path.join(data_dir, 'fra.txt'), 'r') as f:
        return f.read()

raw_text = read_data_nmt()
print(raw_text[:75])
```

```
Downloading ../data/fra-eng.zip from http://d2l-data.s3-accelerate.amazonaws.com/fra-eng.zip.
↪...
Go. Va !
Hi. Salut !
Run!      Cours !
Run!      Courez !
Who?      Qui ?
Wow!      Ça alors !
```

After downloading the dataset, we proceed with several preprocessing steps for the raw text data. For instance, we replace non-breaking space with space, convert uppercase letters to lowercase ones, and insert space between words and punctuation marks.

```
#@save
def preprocess_nmt(text):
    """Preprocess the English-French dataset."""
    def no_space(char, prev_char):
        return char in set(',.!?') and prev_char != ' '

    # Replace non-breaking space with space, and convert uppercase letters to
    # lowercase ones
    text = text.replace('\u202f', ' ').replace('\xa0', ' ').lower()
    # Insert space between words and punctuation marks
    out = [' ' + char if i > 0 and no_space(char, text[i - 1]) else char
```

(continues on next page)

¹¹⁴ <http://www.manythings.org/anki/>

```

        for i, char in enumerate(text)]
    return ''.join(out)

text = preprocess_nmt(raw_text)
print(text[:80])

```

```

go .      va !
hi .      salut !
run !     cours !
run !     courez !
who ?     qui ?
wow !     ça alors !

```

9.5.2 Tokenization

Different from character-level tokenization in [Section 8.3](#), for machine translation we prefer word-level tokenization here (state-of-the-art models may use more advanced tokenization techniques). The following `tokenize_nmt` function tokenizes the the first `num_examples` text sequence pairs, where each token is either a word or a punctuation mark. This function returns two lists of token lists: `source` and `target`. Specifically, `source[i]` is a list of tokens from the i^{th} text sequence in the source language (English here) and `target[i]` is that in the target language (French here).

```

#@save
def tokenize_nmt(text, num_examples=None):
    """Tokenize the English-French dataset."""
    source, target = [], []
    for i, line in enumerate(text.split('\n')):
        if num_examples and i > num_examples:
            break
        parts = line.split('\t')
        if len(parts) == 2:
            source.append(parts[0].split(' '))
            target.append(parts[1].split(' '))
    return source, target

source, target = tokenize_nmt(text)
source[:6], target[:6]

```

```

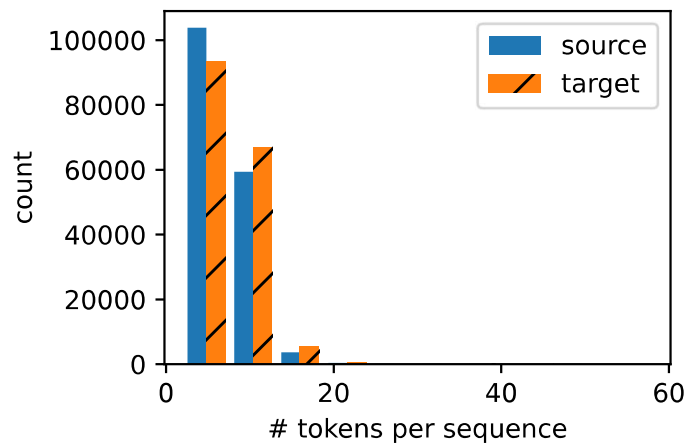
([['go', '.'],
 ['hi', '.'],
 ['run', '!'],
 ['run', '!'],
 ['who', '?'],
 ['wow', '!']],
 [['va', '!'],
 ['salut', '!'],
 ['cours', '!'],
 ['courez', '!'],
 ['qui', '?'],
 ['ça', 'alors', '!']])

```

Let us plot the histogram of the number of tokens per text sequence. In this simple English-French dataset, most of the text sequences have fewer than 20 tokens.

```
#@save
def show_list_len_pair_hist(legend, xlabel, ylabel, xlist, ylist):
    """Plot the histogram for list length pairs."""
    d2l.set_figsize()
    _, _, patches = d2l.plt.hist(
        [[len(l) for l in xlist], [len(l) for l in ylist]])
    d2l.plt.xlabel(xlabel)
    d2l.plt.ylabel(ylabel)
    for patch in patches[1].patches:
        patch.set_hatch('/')
    d2l.plt.legend(legend)

show_list_len_pair_hist(['source', 'target'], '# tokens per sequence',
                        'count', source, target);
```



9.5.3 Vocabulary

Since the machine translation dataset consists of pairs of languages, we can build two vocabularies for both the source language and the target language separately. With word-level tokenization, the vocabulary size will be significantly larger than that using character-level tokenization. To alleviate this, here we treat infrequent tokens that appear less than 2 times as the same unknown (“<unk>”) token. Besides that, we specify additional special tokens such as for padding (“<pad>”) sequences to the same length in minibatches, and for marking the beginning (“<bos>”) or end (“<eos>”) of sequences. Such special tokens are commonly used in natural language processing tasks.

```
src_vocab = d2l.Vocab(source, min_freq=2,
                      reserved_tokens=['<pad>', '<bos>', '<eos>'])
len(src_vocab)
```

```
10012
```

9.5.4 Reading the Dataset

Recall that in language modeling each sequence example, either a segment of one sentence or a span over multiple sentences, has a fixed length. This was specified by the `num_steps` (number of time steps or tokens) argument in [Section 8.3](#). In machine translation, each example is a pair of source and target text sequences, where each text sequence may have different lengths.

For computational efficiency, we can still process a minibatch of text sequences at one time by *truncation* and *padding*. Suppose that every sequence in the same minibatch should have the same length `num_steps`. If a text sequence has fewer than `num_steps` tokens, we will keep appending the special “<pad>” token to its end until its length reaches `num_steps`. Otherwise, we will truncate the text sequence by only taking its first `num_steps` tokens and discarding the remaining. In this way, every text sequence will have the same length to be loaded in minibatches of the same shape.

The following `truncate_pad` function truncates or pads text sequences as described before.

```
#@save
def truncate_pad(line, num_steps, padding_token):
    """Truncate or pad sequences."""
    if len(line) > num_steps:
        return line[:num_steps] # Truncate
    return line + [padding_token] * (num_steps - len(line)) # Pad

truncate_pad(src_vocab[source[0]], 10, src_vocab['<pad>'])
```

```
[47, 4, 1, 1, 1, 1, 1, 1, 1, 1]
```

Now we define a function to transform text sequences into minibatches for training. We append the special “<eos>” token to the end of every sequence to indicate the end of the sequence. When a model is predicting by generating a sequence token after token, the generation of the “<eos>” token can suggest that the output sequence is complete. Besides, we also record the length of each text sequence excluding the padding tokens. This information will be needed by some models that we will cover later.

```
#@save
def build_array_nmt(lines, vocab, num_steps):
    """Transform text sequences of machine translation into minibatches."""
    lines = [vocab[l] for l in lines]
    lines = [l + [vocab['<eos>']] for l in lines]
    array = torch.tensor([truncate_pad(
        l, num_steps, vocab['<pad>']) for l in lines])
    valid_len = (array != vocab['<pad>']).type(torch.int32).sum(1)
    return array, valid_len
```

9.5.5 Putting All Things Together

Finally, we define the `load_data_nmt` function to return the data iterator, together with the vocabularies for both the source language and the target language.

```
#@save
def load_data_nmt(batch_size, num_steps, num_examples=600):
    """Return the iterator and the vocabularies of the translation dataset."""
    text = preprocess_nmt(read_data_nmt())
    source, target = tokenize_nmt(text, num_examples)
    src_vocab = d2l.Vocab(source, min_freq=2,
                          reserved_tokens=['<pad>', '<bos>', '<eos>'])
    tgt_vocab = d2l.Vocab(target, min_freq=2,
                          reserved_tokens=['<pad>', '<bos>', '<eos>'])
    src_array, src_valid_len = build_array_nmt(source, src_vocab, num_steps)
    tgt_array, tgt_valid_len = build_array_nmt(target, tgt_vocab, num_steps)
    data_arrays = (src_array, src_valid_len, tgt_array, tgt_valid_len)
    data_iter = d2l.load_array(data_arrays, batch_size)
    return data_iter, src_vocab, tgt_vocab
```

Let us read the first minibatch from the English-French dataset.

```
train_iter, src_vocab, tgt_vocab = load_data_nmt(batch_size=2, num_steps=8)
for X, X_valid_len, Y, Y_valid_len in train_iter:
    print('X:', X.type(torch.int32))
    print('valid lengths for X:', X_valid_len)
    print('Y:', Y.type(torch.int32))
    print('valid lengths for Y:', Y_valid_len)
    break
```

```
X: tensor([[ 9, 72,  4,  3,  1,  1,  1,  1],
           [ 9, 28,  4,  3,  1,  1,  1,  1]], dtype=torch.int32)
valid lengths for X: tensor([4, 4])
Y: tensor([[ 0, 55,  4,  3,  1,  1,  1,  1],
           [59,  5,  3,  1,  1,  1,  1,  1]], dtype=torch.int32)
valid lengths for Y: tensor([4, 3])
```

Summary

- Machine translation refers to the automatic translation of a sequence from one language to another.
- Using word-level tokenization, the vocabulary size will be significantly larger than that using character-level tokenization. To alleviate this, we can treat infrequent tokens as the same unknown token.
- We can truncate and pad text sequences so that all of them will have the same length to be loaded in minibatches.

Exercises

1. Try different values of the `num_examples` argument in the `load_data_nmt` function. How does this affect the vocabulary sizes of the source language and the target language?
2. Text in some languages such as Chinese and Japanese does not have word boundary indicators (e.g., space). Is word-level tokenization still a good idea for such cases? Why or why not?

9.6 Encoder-Decoder Architecture

As we have discussed in [Section 9.5](#), machine translation is a major problem domain for sequence transduction models, whose input and output are both variable-length sequences. To handle this type of inputs and outputs, we can design an architecture with two major components. The first component is an *encoder*: it takes a variable-length sequence as the input and transforms it into a state with a fixed shape. The second component is a *decoder*: it maps the encoded state of a fixed shape to a variable-length sequence. This is called an *encoder-decoder* architecture, which is depicted in [Fig. 9.6.1](#).



Fig. 9.6.1: The encoder-decoder architecture.

Let us take machine translation from English to French as an example. Given an input sequence in English: “They”, “are”, “watching”, “.”, this encoder-decoder architecture first encodes the variable-length input into a state, then decodes the state to generate the translated sequence token by token as the output: “Ils”, “regardent”, “.”. Since the encoder-decoder architecture forms the basis of different sequence transduction models in subsequent sections, this section will convert this architecture into an interface that will be implemented later.

9.6.1 Encoder

In the encoder interface, we just specify that the encoder takes variable-length sequences as the input `x`. The implementation will be provided by any model that inherits this base `Encoder` class.

```
from torch import nn

#@save
class Encoder(nn.Module):
    """The base encoder interface for the encoder-decoder architecture."""
    def __init__(self, **kwargs):
        super(Encoder, self).__init__(**kwargs)
```

(continues on next page)

```
def forward(self, X, *args):
    raise NotImplementedError
```

9.6.2 Decoder

In the following decoder interface, we add an additional `init_state` function to convert the encoder output (`enc_outputs`) into the encoded state. Note that this step may need extra inputs such as the valid length of the input, which was explained in [Section 9.5.4](#). To generate a variable-length sequence token by token, every time the decoder may map an input (e.g., the generated token at the previous time step) and the encoded state into an output token at the current time step.

```
#@save
class Decoder(nn.Module):
    """The base decoder interface for the encoder-decoder architecture."""
    def __init__(self, **kwargs):
        super(Decoder, self).__init__(**kwargs)

    def init_state(self, enc_outputs, *args):
        raise NotImplementedError

    def forward(self, X, state):
        raise NotImplementedError
```

9.6.3 Putting the Encoder and Decoder Together

In the end, the encoder-decoder architecture contains both an encoder and a decoder, with optionally extra arguments. In the forward propagation, the output of the encoder is used to produce the encoded state, and this state will be further used by the decoder as one of its input.

```
#@save
class EncoderDecoder(nn.Module):
    """The base class for the encoder-decoder architecture."""
    def __init__(self, encoder, decoder, **kwargs):
        super(EncoderDecoder, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, enc_X, dec_X, *args):
        enc_outputs = self.encoder(enc_X, *args)
        dec_state = self.decoder.init_state(enc_outputs, *args)
        return self.decoder(dec_X, dec_state)
```

The term “state” in the encoder-decoder architecture has probably inspired you to implement this architecture using neural networks with states. In the next section, we will see how to apply RNNs to design sequence transduction models based on this encoder-decoder architecture.

Summary

- The encoder-decoder architecture can handle inputs and outputs that are both variable-length sequences, thus is suitable for sequence transduction problems such as machine translation.
- The encoder takes a variable-length sequence as the input and transforms it into a state with a fixed shape.
- The decoder maps the encoded state of a fixed shape to a variable-length sequence.

Exercises

1. Suppose that we use neural networks to implement the encoder-decoder architecture. Do the encoder and the decoder have to be the same type of neural network?
2. Besides machine translation, can you think of another application where the encoder-decoder architecture can be applied?

9.7 Sequence to Sequence Learning

As we have seen in [Section 9.5](#), in machine translation both the input and output are a variable-length sequence. To address this type of problem, we have designed a general encoder-decoder architecture in [Section 9.6](#). In this section, we will use two RNNs to design the encoder and the decoder of this architecture and apply it to *sequence to sequence* learning for machine translation ([Sutskever et al., 2014](#); [Cho et al., 2014b](#)).

Following the design principle of the encoder-decoder architecture, the RNN encoder can take a variable-length sequence as the input and transforms it into a fixed-shape hidden state. In other words, information of the input (source) sequence is *encoded* in the hidden state of the RNN encoder. To generate the output sequence token by token, a separate RNN decoder can predict the next token based on what tokens have been seen (such as in language modeling) or generated, together with the encoded information of the input sequence. [Fig. 9.7.1](#) illustrates how to use two RNNs for sequence to sequence learning in machine translation.

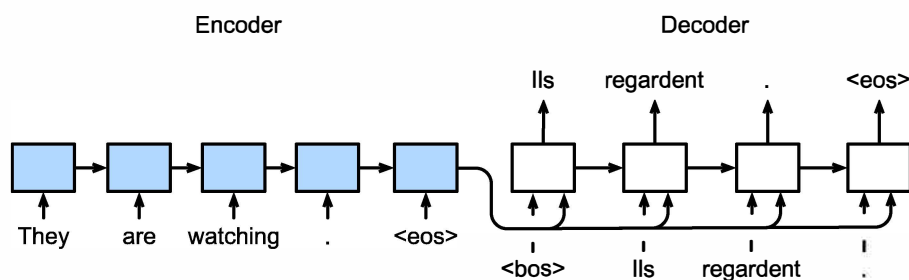


Fig. 9.7.1: Sequence to sequence learning with an RNN encoder and an RNN decoder.

In Fig. 9.7.1, the special “<eos>” token marks the end of the sequence. The model can stop making predictions once this token is generated. At the initial time step of the RNN decoder, there are two special design decisions. First, the special beginning-of-sequence “<bos>” token is an input. Second, the final hidden state of the RNN encoder is used to initiate the hidden state of the decoder. In designs such as (Sutskever et al., 2014), this is exactly how the encoded input sequence information is fed into the decoder for generating the output (target) sequence. In some other designs such as (Cho et al., 2014b), the final hidden state of the encoder is also fed into the decoder as part of the inputs at every time step as shown in Fig. 9.7.1. Similar to the training of language models in Section 8.3, we can allow the labels to be the original output sequence, shifted by one token: “<bos>”, “Ils”, “regardent”, “.” → “Ils”, “regardent”, “.”, “<eos>”.

In the following, we will explain the design of Fig. 9.7.1 in greater detail. We will train this model for machine translation on the English-French dataset as introduced in Section 9.5.

```
import collections
import math
import torch
from torch import nn
from d2l import torch as d2l
```

9.7.1 Encoder

Technically speaking, the encoder transforms an input sequence of variable length into a fixed-shape *context variable* $\mathbf{c}$, and encodes the input sequence information in this context variable. As depicted in Fig. 9.7.1, we can use an RNN to design the encoder.

Let us consider a sequence example (batch size: 1). Suppose that the input sequence is $x_1, \dots, x_T$, such that x_t is the t^{th} token in the input text sequence. At time step t , the RNN transforms the input feature vector $\mathbf{x}_t$ for x_t and the hidden state $\mathbf{h}_{t-1}$ from the previous time step into the current hidden state $\mathbf{h}_t$. We can use a function f to express the transformation of the RNN’s recurrent layer:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}). \quad (9.7.1)$$

In general, the encoder transforms the hidden states at all the time steps into the context variable through a customized function q :

$$\mathbf{c} = q(\mathbf{h}_1, \dots, \mathbf{h}_T). \quad (9.7.2)$$

For example, when choosing $q(\mathbf{h}_1, \dots, \mathbf{h}_T) = \mathbf{h}_T$ such as in Fig. 9.7.1, the context variable is just the hidden state $\mathbf{h}_T$ of the input sequence at the final time step.

So far we have used a unidirectional RNN to design the encoder, where a hidden state only depends on the input subsequence at and before the time step of the hidden state. We can also construct encoders using bidirectional RNNs. In this case, a hidden state depends on the subsequence before and after the time step (including the input at the current time step), which encodes the information of the entire sequence.

Now let us implement the RNN encoder. Note that we use an *embedding layer* to obtain the feature vector for each token in the input sequence. The weight of an embedding layer is a matrix whose number of rows equals to the size of the input vocabulary (`vocab_size`) and number of columns equals to the feature vector’s dimension (`embed_size`). For any input token index i , the embedding layer fetches the i^{th} row (starting from 0) of the weight matrix to return its feature vector. Besides, here we choose a multilayer GRU to implement the encoder.

```

#@save
class Seq2SeqEncoder(d2l.Encoder):
    """The RNN encoder for sequence to sequence learning."""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqEncoder, self).__init__(**kwargs)
        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size, num_hiddens, num_layers,
                          dropout=dropout)

    def forward(self, X, *args):
        # The output `X` shape: ('batch_size', 'num_steps', 'embed_size')
        X = self.embedding(X)
        # In RNN models, the first axis corresponds to time steps
        X = X.permute(1, 0, 2)
        # When state is not mentioned, it defaults to zeros
        output, state = self.rnn(X)
        # `output` shape: ('num_steps', 'batch_size', 'num_hiddens')
        # `state` shape: ('num_layers', 'batch_size', 'num_hiddens')
        return output, state

```

The returned variables of recurrent layers have been explained in [Section 8.6](#). Let us still use a concrete example to illustrate the above encoder implementation. Below we instantiate a two-layer GRU encoder whose number of hidden units is 16. Given a minibatch of sequence inputs X (batch size: 4, number of time steps: 7), the hidden states of the last layer at all the time steps (output return by the encoder's recurrent layers) are a tensor of shape (number of time steps, batch size, number of hidden units).

```

encoder = Seq2SeqEncoder(vocab_size=10, embed_size=8, num_hiddens=16,
                        num_layers=2)
encoder.eval()
X = torch.zeros((4, 7), dtype=torch.long)
output, state = encoder(X)
output.shape

```

```

torch.Size([7, 4, 16])

```

Since a GRU is employed here, the shape of the multilayer hidden states at the final time step is (number of hidden layers, batch size, number of hidden units). If an LSTM is used, memory cell information will also be contained in state.

```

state.shape

```

```

torch.Size([2, 4, 16])

```

9.7.2 Decoder

As we just mentioned, the context variable $\mathbf{c}$ of the encoder's output encodes the entire input sequence $x_1, \dots, x_T$. Given the output sequence $y_1, y_2, \dots, y_{T'}$ from the training dataset, for each time step t' (the symbol differs from the time step t of input sequences or encoders), the probability of the decoder output $y_{t'}$ is conditional on the previous output subsequence $y_1, \dots, y_{t'-1}$ and the context variable $\mathbf{c}$, i.e., $P(y_{t'} \mid y_1, \dots, y_{t'-1}, \mathbf{c})$.

To model this conditional probability on sequences, we can use another RNN as the decoder. At any time step t' on the output sequence, the RNN takes the output $y_{t'-1}$ from the previous time step and the context variable $\mathbf{c}$ as its input, then transforms them and the previous hidden state $\mathbf{s}_{t'-1}$ into the hidden state $\mathbf{s}_{t'}$ at the current time step. As a result, we can use a function g to express the transformation of the decoder's hidden layer:

$$\mathbf{s}_{t'} = g(y_{t'-1}, \mathbf{c}, \mathbf{s}_{t'-1}). \quad (9.7.3)$$

After obtaining the hidden state of the decoder, we can use an output layer and the softmax operation to compute the conditional probability distribution $P(y_{t'} \mid y_1, \dots, y_{t'-1}, \mathbf{c})$ for the output at time step t' .

Following Fig. 9.7.1, when implementing the decoder as follows, we directly use the hidden state at the final time step of the encoder to initialize the hidden state of the decoder. This requires that the RNN encoder and the RNN decoder have the same number of layers and hidden units. To further incorporate the encoded input sequence information, the context variable is concatenated with the decoder input at all the time steps. To predict the probability distribution of the output token, a fully-connected layer is used to transform the hidden state at the final layer of the RNN decoder.

```
class Seq2SeqDecoder(d2l.Decoder):
    """The RNN decoder for sequence to sequence learning."""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqDecoder, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size + num_hiddens, num_hiddens, num_layers,
                          dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, *args):
        return enc_outputs[1]

    def forward(self, X, state):
        # The output 'X' shape: ('num_steps', 'batch_size', 'embed_size')
        X = self.embedding(X).permute(1, 0, 2)
        # Broadcast 'context' so it has the same 'num_steps' as 'X'
        context = state[-1].repeat(X.shape[0], 1, 1)
        X_and_context = torch.cat((X, context), 2)
        output, state = self.rnn(X_and_context, state)
        output = self.dense(output).permute(1, 0, 2)
        # 'output' shape: ('batch_size', 'num_steps', 'vocab_size')
        # 'state' shape: ('num_layers', 'batch_size', 'num_hiddens')
        return output, state
```

To illustrate the implemented decoder, below we instantiate it with the same hyperparameters from the aforementioned encoder. As we can see, the output shape of the decoder becomes (batch

size, number of time steps, vocabulary size), where the last dimension of the tensor stores the predicted token distribution.

```
decoder = Seq2SeqDecoder(vocab_size=10, embed_size=8, num_hiddens=16,
                        num_layers=2)
decoder.eval()
state = decoder.init_state(encoder(X))
output, state = decoder(X, state)
output.shape, state.shape
```

```
(torch.Size([4, 7, 10]), torch.Size([2, 4, 16]))
```

To summarize, the layers in the above RNN encoder-decoder model are illustrated in Fig. 9.7.2.

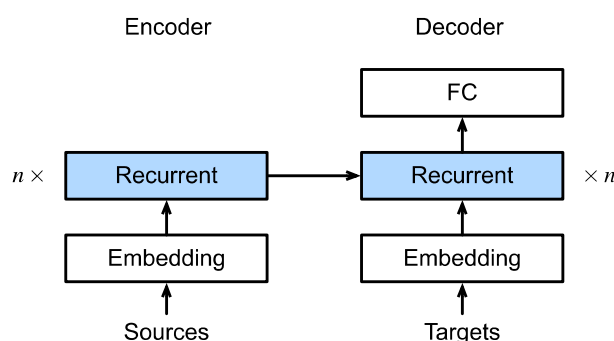


Fig. 9.7.2: Layers in an RNN encoder-decoder model.

9.7.3 Loss Function

At each time step, the decoder predicts a probability distribution for the output tokens. Similar to language modeling, we can apply softmax to obtain the distribution and calculate the cross-entropy loss for optimization. Recall Section 9.5 that the special padding tokens are appended to the end of sequences so sequences of varying lengths can be efficiently loaded in minibatches of the same shape. However, prediction of padding tokens should be excluded from loss calculations.

To this end, we can use the following `sequence_mask` function to mask irrelevant entries with zero values so later multiplication of any irrelevant prediction with zero equals to zero. For example, if the valid length of two sequences excluding padding tokens are one and two, respectively, the remaining entries after the first one and the first two entries are cleared to zeros.

```
##@save
def sequence_mask(X, valid_len, value=0):
    """Mask irrelevant entries in sequences."""
    maxlen = X.size(1)
    mask = torch.arange((maxlen), dtype=torch.float32,
                        device=X.device)[None, :] < valid_len[:, None]
    X[~mask] = value
    return X

X = torch.tensor([[1, 2, 3], [4, 5, 6]])
sequence_mask(X, torch.tensor([1, 2]))
```

```
tensor([[1, 0, 0],
        [4, 5, 0]])
```

We can also mask all the entries across the last few axes. If you like, you may even specify to replace such entries with a non-zero value.

```
X = torch.ones(2, 3, 4)
sequence_mask(X, torch.tensor([1, 2]), value=-1)
```

```
tensor([[[ 1.,  1.,  1.,  1.],
         [-1., -1., -1., -1.],
         [-1., -1., -1., -1.]],
        [[ 1.,  1.,  1.,  1.],
         [ 1.,  1.,  1.,  1.],
         [-1., -1., -1., -1.]])
```

Now we can extend the softmax cross-entropy loss to allow the masking of irrelevant predictions. Initially, masks for all the predicted tokens are set to one. Once the valid length is given, the mask corresponding to any padding token will be cleared to zero. In the end, the loss for all the tokens will be multiplied by the mask to filter out irrelevant predictions of padding tokens in the loss.

```
#!/save
class MaskedSoftmaxCELoss(nn.CrossEntropyLoss):
    """The softmax cross-entropy loss with masks."""
    # `pred` shape: ('batch_size', 'num_steps', 'vocab_size')
    # `label` shape: ('batch_size', 'num_steps')
    # `valid_len` shape: ('batch_size',)
    def forward(self, pred, label, valid_len):
        weights = torch.ones_like(label)
        weights = sequence_mask(weights, valid_len)
        self.reduction='none'
        unweighted_loss = super(MaskedSoftmaxCELoss, self).forward(
            pred.permute(0, 2, 1), label)
        weighted_loss = (unweighted_loss * weights).mean(dim=1)
        return weighted_loss
```

For a sanity check, we can create three identical sequences. Then we can specify that the valid lengths of these sequences are 4, 2, and 0, respectively. As a result, the loss of the first sequence should be twice as large as that of the second sequence, while the third sequence should have a zero loss.

```
loss = MaskedSoftmaxCELoss()
loss(torch.ones(3, 4, 10), torch.ones((3, 4), dtype=torch.long),
      torch.tensor([4, 2, 0]))
```

```
tensor([2.3026, 1.1513, 0.0000])
```

9.7.4 Training

In the following training loop, we concatenate the special beginning-of-sequence token and the original output sequence excluding the final token as the input to the decoder, as shown in Fig. 9.7.1. This is called *teacher forcing* because the original output sequence (token labels) is fed into the decoder. Alternatively, we could also feed the *predicted* token from the previous time step as the current input to the decoder.

```
#@save
def train_seq2seq(net, data_iter, lr, num_epochs, tgt_vocab, device):
    """Train a model for sequence to sequence."""
    def xavier_init_weights(m):
        if type(m) == nn.Linear:
            nn.init.xavier_uniform_(m.weight)
        if type(m) == nn.GRU:
            for param in m._flat_weights_names:
                if "weight" in param:
                    nn.init.xavier_uniform_(m._parameters[param])
    net.apply(xavier_init_weights)
    net.to(device)
    optimizer = torch.optim.Adam(net.parameters(), lr=lr)
    loss = MaskedSoftmaxCELoss()
    net.train()
    animator = d2l.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[10, num_epochs])
    for epoch in range(num_epochs):
        timer = d2l.Timer()
        metric = d2l.Accumulator(2) # Sum of training loss, no. of tokens
        for batch in data_iter:
            optimizer.zero_grad()
            X, X_valid_len, Y, Y_valid_len = [x.to(device) for x in batch]
            bos = torch.tensor([tgt_vocab['<bos>']] * Y.shape[0],
                               device=device).reshape(-1, 1)
            dec_input = torch.cat([bos, Y[:, :-1]], 1) # Teacher forcing
            Y_hat, _ = net(X, dec_input, X_valid_len)
            l = loss(Y_hat, Y, Y_valid_len)
            l.sum().backward() # Make the loss scalar for 'backward'
            d2l.grad_clipping(net, 1)
            num_tokens = Y_valid_len.sum()
            optimizer.step()
            with torch.no_grad():
                metric.add(l.sum(), num_tokens)
        if (epoch + 1) % 10 == 0:
            animator.add(epoch + 1, (metric[0] / metric[1],))
    print(f'loss {metric[0] / metric[1]:.3f}, {metric[1] / timer.stop():.1f} '
          f'tokens/sec on {str(device)}')
```

Now we can create and train an RNN encoder-decoder model for sequence to sequence learning on the machine translation dataset.

```
embed_size, num_hiddens, num_layers, dropout = 32, 32, 2, 0.1
batch_size, num_steps = 64, 10
lr, num_epochs, device = 0.005, 300, d2l.try_gpu()

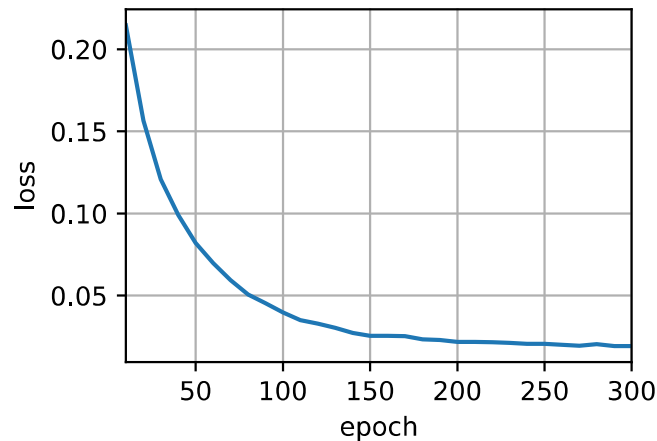
train_iter, src_vocab, tgt_vocab = d2l.load_data_nmt(batch_size, num_steps)
```

(continues on next page)

(continued from previous page)

```
encoder = Seq2SeqEncoder(  
    len(src_vocab), embed_size, num_hiddens, num_layers, dropout)  
decoder = Seq2SeqDecoder(  
    len(tgt_vocab), embed_size, num_hiddens, num_layers, dropout)  
net = d2l.EncoderDecoder(encoder, decoder)  
train_seq2seq(net, train_iter, lr, num_epochs, tgt_vocab, device)
```

```
loss 0.019, 11254.5 tokens/sec on cuda:0
```



9.7.5 Prediction

To predict the output sequence token by token, at each decoder time step the predicted token from the previous time step is fed into the decoder as an input. Similar to training, at the initial time step the beginning-of-sequence (“<bos>”) token is fed into the decoder. This prediction process is illustrated in Fig. 9.7.3. When the end-of-sequence (“<eos>”) token is predicted, the prediction of the output sequence is complete.

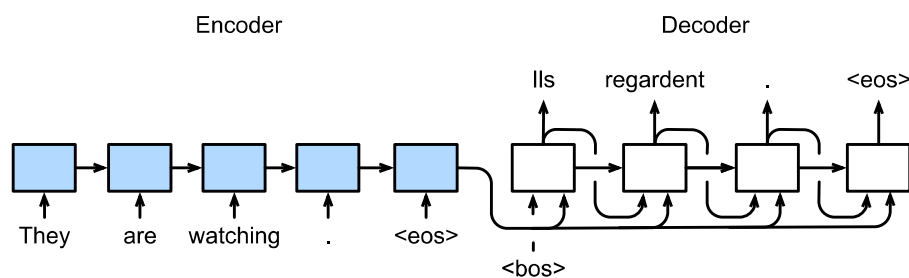


Fig. 9.7.3: Predicting the output sequence token by token using an RNN encoder-decoder.

We will introduce different strategies for sequence generation in [Section 9.8](#).

```
#@save  
def predict_seq2seq(net, src_sentence, src_vocab, tgt_vocab, num_steps,  
    device, save_attention_weights=False):
```

(continues on next page)


```

"""Predict for sequence to sequence."""
# Set `net` to eval mode for inference
net.eval()
src_tokens = src_vocab[src_sentence.lower().split(' ')] + [
    src_vocab['<eos>']]
enc_valid_len = torch.tensor([len(src_tokens)], device=device)
src_tokens = d2l.truncate_pad(src_tokens, num_steps, src_vocab['<pad>'])
# Add the batch axis
enc_X = torch.unsqueeze(
    torch.tensor(src_tokens, dtype=torch.long, device=device), dim=0)
enc_outputs = net.encoder(enc_X, enc_valid_len)
dec_state = net.decoder.init_state(enc_outputs, enc_valid_len)
# Add the batch axis
dec_X = torch.unsqueeze(torch.tensor(
    [tgt_vocab['<bos>']], dtype=torch.long, device=device), dim=0)
output_seq, attention_weight_seq = [], []
for _ in range(num_steps):
    Y, dec_state = net.decoder(dec_X, dec_state)
    # We use the token with the highest prediction likelihood as the input
    # of the decoder at the next time step
    dec_X = Y.argmax(dim=2)
    pred = dec_X.squeeze(dim=0).type(torch.int32).item()
    # Save attention weights (to be covered later)
    if save_attention_weights:
        attention_weight_seq.append(net.decoder.attention_weights)
    # Once the end-of-sequence token is predicted, the generation of the
    # output sequence is complete
    if pred == tgt_vocab['<eos>']:
        break
    output_seq.append(pred)
return ' '.join(tgt_vocab.to_tokens(output_seq)), attention_weight_seq

```

9.7.6 Evaluation of Predicted Sequences

We can evaluate a predicted sequence by comparing it with the label sequence (the ground-truth). BLEU (Bilingual Evaluation Understudy), though originally proposed for evaluating machine translation results (Papineni et al., 2002), has been extensively used in measuring the quality of output sequences for different applications. In principle, for any n -grams in the predicted sequence, BLEU evaluates whether this n -grams appears in the label sequence.

Denote by p_n the precision of n -grams, which is the ratio of the number of matched n -grams in the predicted and label sequences to the number of n -grams in the predicted sequence. To explain, given a label sequence A, B, C, D, E, F , and a predicted sequence A, B, B, C, D , we have $p_1 = 4/5$, $p_2 = 3/4$, $p_3 = 1/3$, and $p_4 = 0$. Besides, let $\text{len}_{\text{label}}$ and len_{pred} be the numbers of tokens in the label sequence and the predicted sequence, respectively. Then, BLEU is defined as

$$\exp \left(\min \left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}} \right) \right) \prod_{n=1}^k p_n^{1/2^n}, \quad (9.7.4)$$

where k is the longest n -grams for matching.

Based on the definition of BLEU in (9.7.4), whenever the predicted sequence is the same as the label sequence, BLEU is 1. Moreover, since matching longer n -grams is more difficult, BLEU assigns

a greater weight to a longer n -gram precision. Specifically, when p_n is fixed, $p_n^{1/2^n}$ increases as n grows (the original paper uses $p_n^{1/n}$). Furthermore, since predicting shorter sequences tends to obtain a higher p_n value, the coefficient before the multiplication term in (9.7.4) penalizes shorter predicted sequences. For example, when $k = 2$, given the label sequence A, B, C, D, E, F and the predicted sequence A, B , although $p_1 = p_2 = 1$, the penalty factor $\exp(1 - 6/2) \approx 0.14$ lowers the BLEU.

We implement the BLEU measure as follows.

```
def bleu(pred_seq, label_seq, k): #@save
    """Compute the BLEU."""
    pred_tokens, label_tokens = pred_seq.split(' '), label_seq.split(' ')
    len_pred, len_label = len(pred_tokens), len(label_tokens)
    score = math.exp(min(0, 1 - len_label / len_pred))
    for n in range(1, k + 1):
        num_matches, label_subs = 0, collections.defaultdict(int)
        for i in range(len_label - n + 1):
            label_subs[' '.join(label_tokens[i: i + n])] += 1
        for i in range(len_pred - n + 1):
            if label_subs[' '.join(pred_tokens[i: i + n])] > 0:
                num_matches += 1
                label_subs[' '.join(pred_tokens[i: i + n])] -= 1
        score *= math.pow(num_matches / (len_pred - n + 1), math.pow(0.5, n))
    return score
```

In the end, we use the trained RNN encoder-decoder to translate a few English sentences into French and compute the BLEU of the results.

```
engs = ['go .', 'i lost .', 'he\'s calm .', 'i\'m home .']
fras = ['va !', 'j\'ai perdu .', 'il est calme .', 'je suis chez moi .']
for eng, fra in zip(engs, fras):
    translation, attention_weight_seq = predict_seq2seq(
        net, eng, src_vocab, tgt_vocab, num_steps, device)
    print(f'{eng} => {translation}, bleu {bleu(translation, fra, k=2):.3f}')
```

```
go . => va !, bleu 1.000
i lost . => j'ai <unk> rouler !, bleu 0.000
he's calm . => il est paresseux prie ., bleu 0.548
i'm home . => je suis chez moi ., bleu 1.000
```

Summary

- Following the design of the encoder-decoder architecture, we can use two RNNs to design a model for sequence to sequence learning.
- When implementing the encoder and the decoder, we can use multilayer RNNs.
- We can use masks to filter out irrelevant computations, such as when calculating the loss.
- In encoder-decoder training, the teacher forcing approach feeds original output sequences (in contrast to predictions) into the decoder.
- BLEU is a popular measure for evaluating output sequences by matching n -grams between the predicted sequence and the label sequence.

Exercises

1. Can you adjust the hyperparameters to improve the translation results?
2. Rerun the experiment without using masks in the loss calculation. What results do you observe? Why?
3. If the encoder and the decoder differ in the number of layers or the number of hidden units, how can we initialize the hidden state of the decoder?
4. In training, replace teacher forcing with feeding the prediction at the previous time step into the decoder. How does this influence the performance?
5. Rerun the experiment by replacing GRU with LSTM.
6. Are there any other ways to design the output layer of the decoder?

9.8 Beam Search

In [Section 9.7](#), we predicted the output sequence token by token until the special end-of-sequence “<eos>” token is predicted. In this section, we will begin with formalizing this *greedy search* strategy and exploring issues with it, then compare this strategy with other alternatives: *exhaustive search* and *beam search*.

Before a formal introduction to greedy search, let us formalize the search problem using the same mathematical notation from [Section 9.7](#). At any time step t' , the probability of the decoder output $y_{t'}$ is conditional on the output subsequence $y_1, \dots, y_{t'-1}$ before t' and the context variable $\mathbf{c}$ that encodes the information of the input sequence. To quantify computational cost, denote by $\mathcal{Y}$ (it contains “<eos>”) the output vocabulary. So the cardinality $|\mathcal{Y}|$ of this vocabulary set is the vocabulary size. Let us also specify the maximum number of tokens of an output sequence as T' . As a result, our goal is to search for an ideal output from all the $\mathcal{O}(|\mathcal{Y}|^{T'})$ possible output sequences. Of course, for all these output sequences, portions including and after “<eos>” will be discarded in the actual output.

9.8.1 Greedy Search

First, let us take a look at a simple strategy: *greedy search*. This strategy has been used to predict sequences in [Section 9.7](#). In greedy search, at any time step t' of the output sequence, we search for the token with the highest conditional probability from $\mathcal{Y}$, i.e.,

$$y_{t'} = \operatorname{argmax}_{y \in \mathcal{Y}} P(y \mid y_1, \dots, y_{t'-1}, \mathbf{c}), \quad (9.8.1)$$

as the output. Once “<eos>” is outputted or the output sequence has reached its maximum length T' , the output sequence is completed.

So what can go wrong with greedy search? In fact, the *optimal sequence* should be the output sequence with the maximum $\prod_{t'=1}^{T'} P(y_{t'} \mid y_1, \dots, y_{t'-1}, \mathbf{c})$, which is the conditional probability of generating an output sequence based on the input sequence. Unfortunately, there is no guarantee that the optimal sequence will be obtained by greedy search.

Time step	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

Fig. 9.8.1: At each time step, greedy search selects the token with the highest conditional probability.

Let us illustrate it with an example. Suppose that there are four tokens “A”, “B”, “C”, and “<eos>” in the output dictionary. In Fig. 9.8.1, the four numbers under each time step represent the conditional probabilities of generating “A”, “B”, “C”, and “<eos>” at that time step, respectively. At each time step, greedy search selects the token with the highest conditional probability. Therefore, the output sequence “A”, “B”, “C”, and “<eos>” will be predicted in Fig. 9.8.1. The conditional probability of this output sequence is $0.5 \times 0.4 \times 0.4 \times 0.6 = 0.048$.

Time step	1	2	3	4
A	0.5	0.1	0.1	0.1
B	0.2	0.4	0.6	0.2
C	0.2	0.3	0.2	0.1
<eos>	0.1	0.2	0.1	0.6

Fig. 9.8.2: The four numbers under each time step represent the conditional probabilities of generating “A”, “B”, “C”, and “<eos>” at that time step. At time step 2, the token “C”, which has the second highest conditional probability, is selected.

Next, let us look at another example in Fig. 9.8.2. Unlike in Fig. 9.8.1, at time step 2 we select the token “C” in Fig. 9.8.2, which has the *second* highest conditional probability. Since the output subsequences at time steps 1 and 2, on which time step 3 is based, have changed from “A” and “B” in Fig. 9.8.1 to “A” and “C” in Fig. 9.8.2, the conditional probability of each token at time step 3 has also changed in Fig. 9.8.2. Suppose that we choose the token “B” at time step 3. Now time step 4 is conditional on the output subsequence at the first three time steps “A”, “C”, and “B”, which is different from “A”, “B”, and “C” in Fig. 9.8.1. Therefore, the conditional probability of generating each token at time step 4 in Fig. 9.8.2 is also different from that in Fig. 9.8.1. As a result, the conditional probability of the output sequence “A”, “C”, “B”, and “<eos>” in Fig. 9.8.2 is $0.5 \times 0.3 \times 0.6 \times 0.6 = 0.054$, which is greater than that of greedy search in Fig. 9.8.1. In this example, the output sequence “A”, “B”, “C”, and “<eos>” obtained by the greedy search is not an optimal sequence.

9.8.2 Exhaustive Search

If the goal is to obtain the optimal sequence, we may consider using *exhaustive search*: exhaustively enumerate all the possible output sequences with their conditional probabilities, then output the one with the highest conditional probability.

Although we can use exhaustive search to obtain the optimal sequence, its computational cost $\mathcal{O}(|\mathcal{Y}|^{T'})$ is likely to be excessively high. For example, when $|\mathcal{Y}| = 10000$ and $T' = 10$, we will need to evaluate $10000^{10} = 10^{40}$ sequences. This is next to impossible! On the other hand, the computational cost of greedy search is $\mathcal{O}(|\mathcal{Y}| T')$: it is usually significantly smaller than that of exhaustive search. For example, when $|\mathcal{Y}| = 10000$ and $T' = 10$, we only need to evaluate $10000 \times 10 = 10^5$ sequences.

9.8.3 Beam Search

Decisions about sequence searching strategies lie on a spectrum, with easy questions at either extreme. What if only accuracy matters? Obviously, exhaustive search. What if only computational cost matters? Clearly, greedy search. A real-world application usually asks a complicated question, somewhere in between those two extremes.

Beam search is an improved version of greedy search. It has a hyperparameter named *beam size*, k . At time step 1, we select k tokens with the highest conditional probabilities. Each of them will be the first token of k candidate output sequences, respectively. At each subsequent time step, based on the k candidate output sequences at the previous time step, we continue to select k candidate output sequences with the highest conditional probabilities from $k|\mathcal{Y}|$ possible choices.

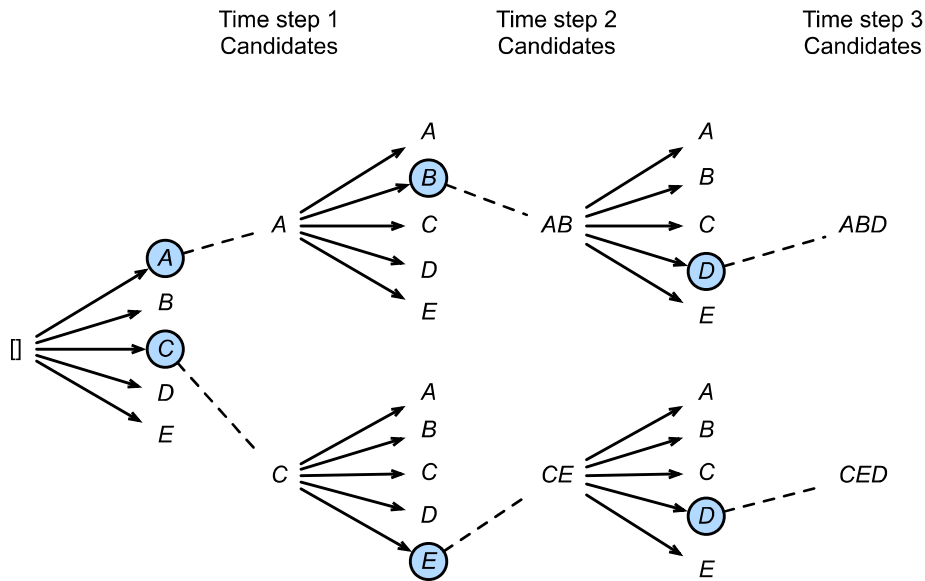


Fig. 9.8.3: The process of beam search (beam size: 2, maximum length of an output sequence: 3). The candidate output sequences are A, C, AB, CE, ABD, and CED.

Fig. 9.8.3 demonstrates the process of beam search with an example. Suppose that the output vocabulary contains only five elements: $\mathcal{Y} = \{A, B, C, D, E\}$, where one of them is “<eos>”. Let the beam size be 2 and the maximum length of an output sequence be 3. At time step 1, suppose

that the tokens with the highest conditional probabilities $P(y_1 | \mathbf{c})$ are A and C . At time step 2, for all $y_2 \in \mathcal{Y}$, we compute

$$\begin{aligned} P(A, y_2 | \mathbf{c}) &= P(A | \mathbf{c})P(y_2 | A, \mathbf{c}), \\ P(C, y_2 | \mathbf{c}) &= P(C | \mathbf{c})P(y_2 | C, \mathbf{c}), \end{aligned} \quad (9.8.2)$$

and pick the largest two among these ten values, say $P(A, B | \mathbf{c})$ and $P(C, E | \mathbf{c})$. Then at time step 3, for all $y_3 \in \mathcal{Y}$, we compute

$$\begin{aligned} P(A, B, y_3 | \mathbf{c}) &= P(A, B | \mathbf{c})P(y_3 | A, B, \mathbf{c}), \\ P(C, E, y_3 | \mathbf{c}) &= P(C, E | \mathbf{c})P(y_3 | C, E, \mathbf{c}), \end{aligned} \quad (9.8.3)$$

and pick the largest two among these ten values, say $P(A, B, D | \mathbf{c})$ and $P(C, E, D | \mathbf{c})$. As a result, we get six candidate output sequences: (i) A ; (ii) C ; (iii) A, B ; (iv) C, E ; (v) A, B, D ; and (vi) C, E, D .

In the end, we obtain the set of final candidate output sequences based on these six sequences (e.g., discard portions including and after “<eos>”). Then we choose the sequence with the highest of the following score as the output sequence:

$$\frac{1}{L^\alpha} \log P(y_1, \dots, y_L | \mathbf{c}) = \frac{1}{L^\alpha} \sum_{t'=1}^L \log P(y_{t'} | y_1, \dots, y_{t'-1}, \mathbf{c}), \quad (9.8.4)$$

where L is the length of the final candidate sequence and α is usually set to 0.75. Since a longer sequence has more logarithmic terms in the summation of (9.8.4), the term L^α in the denominator penalizes long sequences.

The computational cost of beam search is $\mathcal{O}(k|\mathcal{Y}|T')$. This result is in between that of greedy search and that of exhaustive search. In fact, greedy search can be treated as a special type of beam search with a beam size of 1. With a flexible choice of the beam size, beam search provides a tradeoff between accuracy versus computational cost.

Summary

- Sequence searching strategies include greedy search, exhaustive search, and beam search.
- Beam search provides a tradeoff between accuracy versus computational cost via its flexible choice of the beam size.

Exercises

1. Can we treat exhaustive search as a special type of beam search? Why or why not?
2. Apply beam search in the machine translation problem in [Section 9.7](#). How does the beam size affect the translation results and the prediction speed?
3. We used language modeling for generating text following user-provided prefixes in [Section 8.5](#). Which kind of search strategy does it use? Can you improve it?